

Docker for ML Workloads

docker

Overview

Docker packages an application and its runtime dependencies into a portable container image. For ML work, that usually means Python dependencies, native libraries, model server processes, and sometimes CUDA aligned stacks.

Containerization matters because ML code that works on one laptop can fail elsewhere due to library versions, system packages, or GPU compatibility. Docker reduces that drift by making the runtime explicit.

Core Concepts

- A Docker image is a layered filesystem built from a Dockerfile, while a container is a running instance created from that image.
- Good ML container design separates dependency installation, application code, model loading behavior, and configuration so rebuilds stay fast and reproducible.
- Images are typically stored in a registry such as ECR and then executed by a runtime platform such as ECS, SageMaker, or Kubernetes.

How It Works

- You define a Dockerfile, build the image locally or in CI, run tests inside the image, and publish the versioned artifact to a registry.
- At deployment time, the target platform pulls the image, injects configuration and credentials, and starts the container according to the workload type.
- For inference systems, startup behavior matters because model download time, warmup steps, and dependency weight directly affect deployment speed and cold start characteristics.

AI and ML Relevance

- Docker is the standard packaging boundary for custom training jobs, model servers, feature services, and batch processing components in modern ML platforms.
- It also makes local reproduction easier because the same image can be tested before it is pushed into a managed AWS runtime.

Operational Notes

- Keep images small and deterministic. Bloated images increase build time, push time, scan time, and startup latency.
- Pin dependency versions, scan for vulnerabilities, and separate secrets from the image itself. Secrets belong in runtime configuration, not baked into layers.
- Be especially careful with GPU stacks. CUDA, driver expectations, and framework versions need to line up with the runtime environment.

What To Remember

- Docker solves packaging and runtime consistency, not orchestration or model governance.
- A reproducible image is a prerequisite for reliable ML deployment, not the whole deployment story.
- If the image cannot be rebuilt from source with the same dependencies, release confidence is weaker than it looks.